PROJECT REPORT OF EXPLORATORY PROJECT (EP)

ON

ECG Arrhythmia Detection System Using CNN

LAB PRACTICAL REPORT

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Rajat Sharma (2210990707)

Palak Singla (2210990994)

Pareen Khirbat (2210990639)

Supervised By:

Dr. Gurpreet Singh

Chitkara University Institute of Engineering and Technology

Punjab, India

Academic Session 2025-26

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

DECLARATION

We hereby certify that the work which is being presented in the project report entitled "ECG Arrhythmia Detection System Using CNN" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of Dr. Gurpreet Singh. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Date: ……..

Name: Rajat Sharma (2210990707)
Signature:

Name: Palak Singla (2210990994)
Signature:

Name: Pareen Khirbat (2210990639)
Signature:

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Dr. Gurpreet Singh
Associate Professor
Department of Computer Science and Engineering,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We are sincerely thankful to Dr. Gurpreet Singh for his guidance, feedback, and steady support throughout this project. His suggestions helped us keep the work practical and organized. We also thank Chitkara University Institute of Engineering and Technology for providing the environment and resources needed to complete the report. We are equally grateful to our classmates and friends for the useful discussions we had during development, and to our families for their encouragement and patience.

LIST OF FIGURES AND TABLES

ABSTRACT

This project presents an ECG arrhythmia detection system built around a one-dimensional convolutional neural network and a web-based dashboard. The codebase brings together signal preprocessing, model inference, confidence estimation, uncertainty reporting, waveform visualization, and PDF report generation in one place.

A user can paste ECG samples, upload a CSV or JSON file, or load a demo signal through the frontend. After that, the backend cleans the signal by resampling it, applying band-pass filtering, removing baseline drift, normalizing the values, detecting R-peaks, and preparing beats for prediction.

The prediction output includes the class label, risk score, top class probabilities, signal quality, uncertainty estimate, and an explanation map that is shown on the waveform chart. We kept the workflow simple enough for students to follow, while still making it feel like a real end-to-end AI system.

1. Introduction

Electrocardiogram analysis is one of the most useful tools for early cardiac diagnosis. ECG signals carry patterns that reflect the electrical activity of the heart, and even a small change in shape, timing, or rhythm may point to a health concern. In practice, ECG interpretation is not always easy because signal quality can vary due to movement, electrode placement, or background noise. That is why automated screening systems are helpful: they can perform repeated analysis quickly while still leaving the final decision to a human expert.

We built this project as a full-stack application that combines machine learning with a medical-style interface. Instead of stopping at a model notebook, the codebase includes a backend API, preprocessing utilities, deep learning models, classical model support, uncertainty estimation, and a React-based dashboard. This makes the project more useful for demonstration because the signal can be entered, cleaned, predicted, and displayed in a single workflow.

Another advantage of this setup is that it works well for both technical and presentation purposes. On the research side, it uses standard signal processing steps such as band-pass filtering, baseline correction, and beat segmentation. On the presentation side, it shows a clear dashboard with waveform plots, risk indicators, probability breakdowns, and written explanations. The result is a system that is easy to understand for a non-expert user while still showing how the prediction was made.

2. Methodology

2.1 Problem Statement and Objectives

In clinical environments, ECG data is usually reviewed manually or with specialized software. Even when automatic tools are available, many of them are either too technical for a student-level project or too limited to explain how the prediction was reached. A common issue in academic work is that the model is trained separately from the interface, which leaves the final system incomplete. Another issue is that many prototype systems return only a class label, so the user cannot understand why that result was produced. Our project addresses these gaps by building a complete ECG arrhythmia detection pipeline. The aim was to keep the project practical enough for a lab setup and clear enough to explain during a viva or presentation.

2.2 Related Work and Motivation

Earlier ECG classification methods mainly depended on hand-crafted features, threshold-based detection, and conventional machine learning models. These approaches can work well for a limited task, but they usually rely a lot on feature engineering. If the signal pattern changes or the dataset becomes larger, performance may drop unless the features are redesigned. Deep learning gives a more flexible approach because the model can learn useful waveform patterns directly from the data. One-dimensional convolutional neural networks are especially suitable for ECG analysis because they can capture local temporal patterns such as QRS complexes, spikes, and rhythm variations.

2.3 Dataset Description and Data Handling

The repository is set up to support ECG data from more than one format. The backend can work with MIT-BIH heartbeat categorization files, PTBDB records, PTB-XL data, and engineered feature datasets stored in the Datasets directory. This makes the project more flexible than a single-dataset notebook, because it can be trained or tested on different sources when the files are available. The input validation logic expects at least 100 ECG samples before prediction. We kept that limit on purpose because very short signals are not enough for the preprocessing pipeline or for reliable rhythm inference.

2.4 ECG Preprocessing Pipeline

The preprocessing stage first converts the input into a NumPy array and removes non-finite values so that corrupted points do not affect later steps. After that, the signal is resampled to the target rate of 250 Hz using polyphase resampling. A band-pass filter between 0.5 Hz and 40 Hz is

then applied to reduce slow drift and high-frequency noise while preserving the important ECG shape. Baseline wander is removed using a two-stage median filter, and the signal is normalized with z-score scaling. The project then estimates R-peaks using a Pan-Tompkins-style approach that includes differentiation, squaring, moving-window integration, and adaptive thresholding. When the signal is noisy, the code falls back to a percentile-based threshold, which makes the process more robust.

2.5 Proposed System Architecture

The system follows a three-layer structure made up of the frontend, the backend, and the model layer. The frontend handles user interaction, the backend takes care of file parsing, preprocessing, and request validation, and the model layer performs inference and explanation generation. This separation is useful because each part can be tested independently and improved without disturbing the rest of the system. The architecture also includes an inference engine that tries to load trained deep and classical models. When trained checkpoints are available, the engine uses calibrated predictions and Monte Carlo dropout to estimate uncertainty.

2.6 CNN Model Design and Training Strategy

The codebase uses a multi-head deep learning setup. Each model shares a feature extractor and then splits into separate heads for binary classification, multiclass arrhythmia classification, and signal quality prediction. This design lets the system answer more than one question from the same ECG input. The repository includes three deep models: an Inception-ResNet style network, a CNN plus bidirectional LSTM, and a CNN plus Transformer encoder. These models are wrapped inside an ensemble mechanism with configurable weights. During inference, the predictions from each model are averaged after calibration so that the final output stays more stable.

3. Tools and Technologies

The project was developed using FastAPI for the backend API, React for the user interface, PyTorch for deep learning inference, SciPy for signal processing, and ReportLab for PDF report generation. NumPy and Pandas were used for handling and preparing the data, while Recharts and chart-based components were used to present ECG waveforms and class probabilities in a readable way.

These tools were chosen because they support a clean end-to-end workflow. FastAPI keeps the request logic simple and fast, React provides a responsive dashboard, PyTorch supports model experimentation, and ReportLab helps convert the final prediction into a downloadable report. The modular structure also makes it easier to extend the project later with new datasets, a stronger model, or real-time device integration.

Table 1: Model heads used in the code

Model head	Purpose
Binary head	Normal vs arrhythmia decision
Multiclass head	Specific rhythm classification
Quality head	Good, noisy, unusable signal assessment

4. Implementation

The backend acts as the control center of the application. It accepts ECG data, converts it into a clean numerical form, sends it to the inference engine, and returns structured results. Separate helper methods handle CSV parsing, JSON parsing, prediction response formatting, and PDF report retrieval. The /predict endpoint is the main API route. It expects either signal or signal_2d data, checks the minimum length requirement, and passes the input through the preprocessing pipeline.

The /upload endpoint stores uploaded ECG files and also tries to count the number of numeric points in them. The /report endpoint generates a PDF summary from the prediction result, while /report/{report_id} allows the report to be downloaded later. A health endpoint is also available for quick server checks. These endpoints give the system a proper application structure and make it suitable for front-end integration and external testing tools.

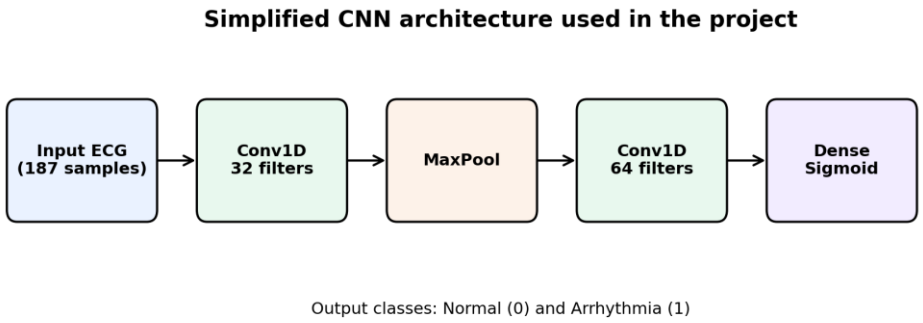


Figure 1: Simplified CNN architecture used in the project

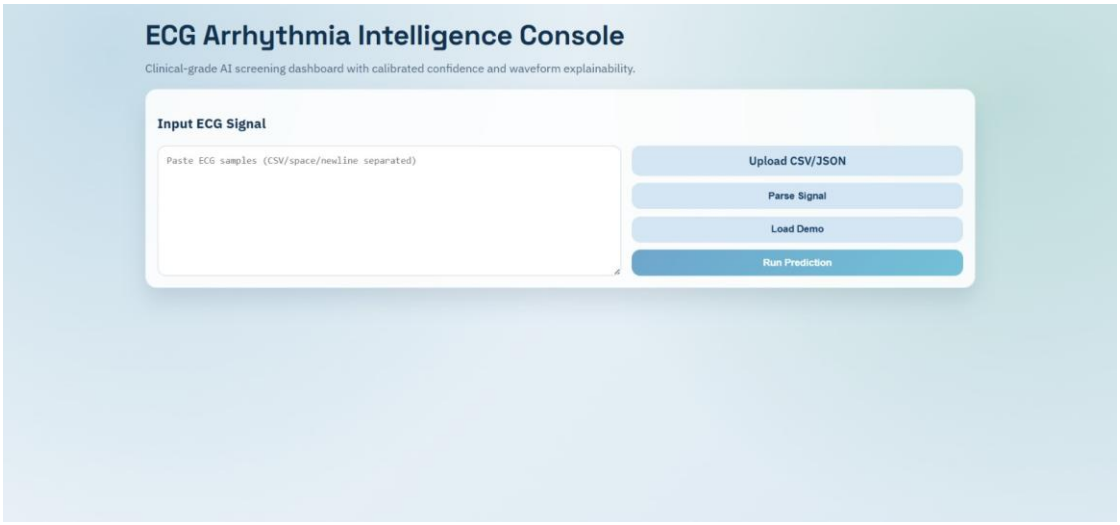


Figure 2: ECG input dashboard with parsed signal data

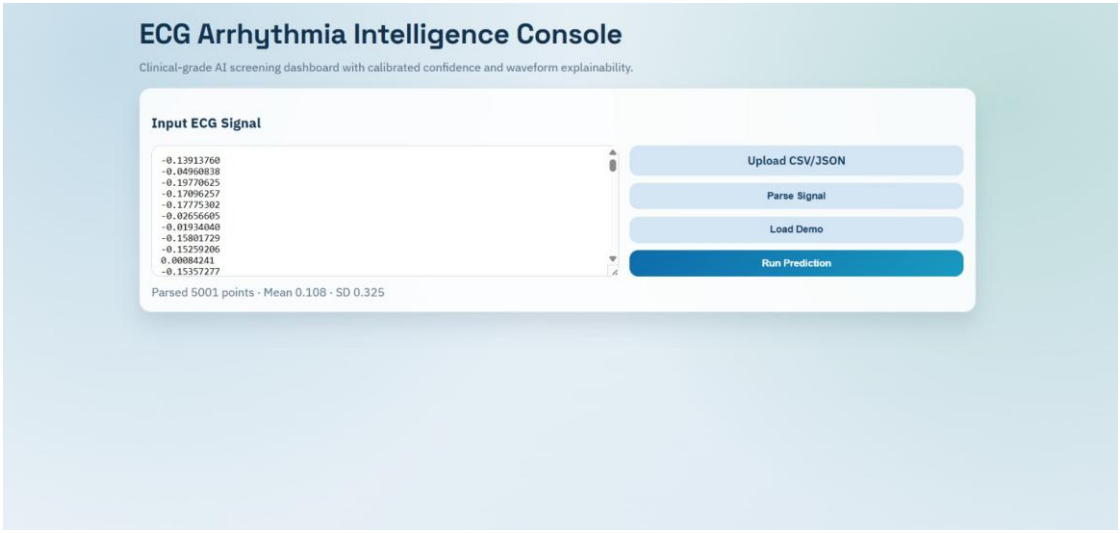


Figure 3: Input panel showing upload, parse, demo, and prediction controls

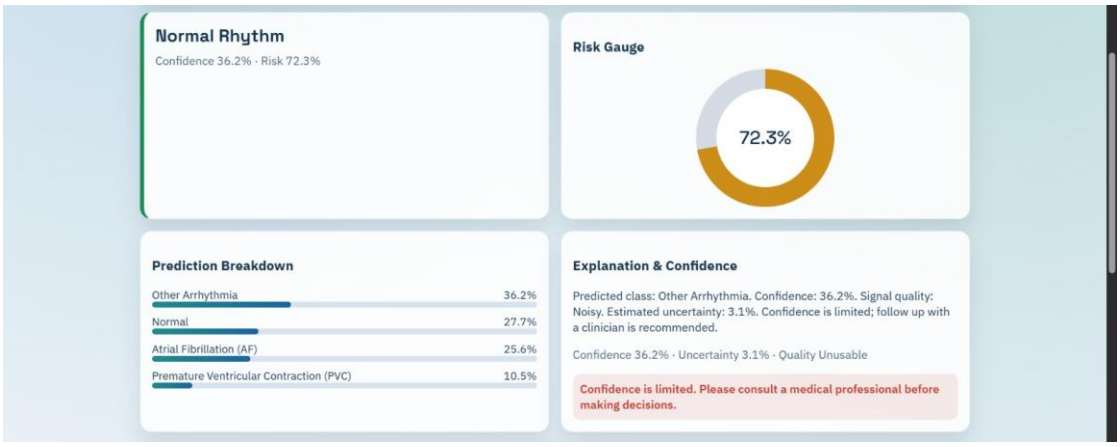


Figure 4: ECG waveform with abnormal highlights

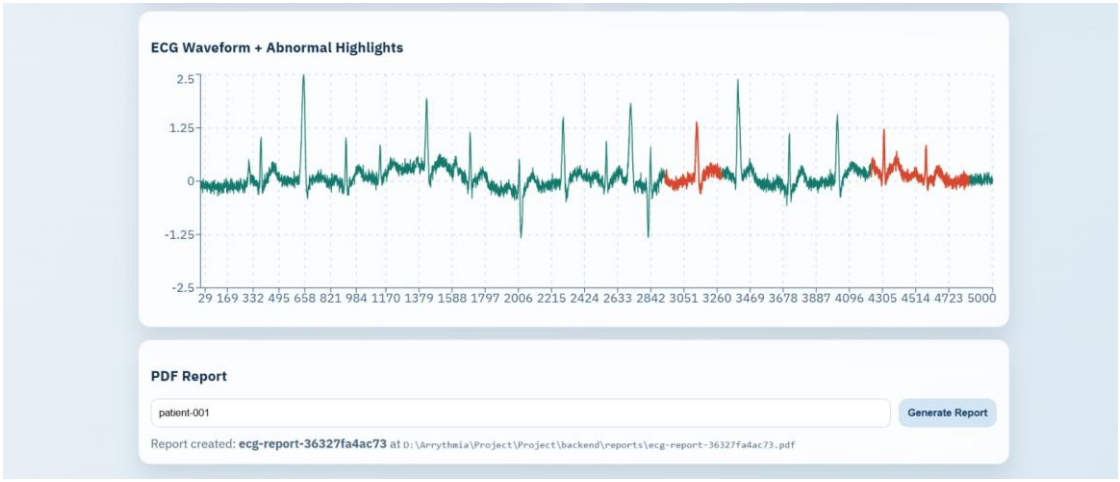


Figure 5: Prediction dashboard with risk gauge, class probabilities, and explanation panel

Table 2: Main backend endpoints

Endpoint	Purpose
POST /predict	Runs preprocessing and ECG classification
POST /upload	Uploads CSV or JSON ECG files
POST /report	Generates a PDF prediction report
GET /report/{report_id}	Downloads an existing report
GET /health	Checks server status

5. Result

After the prediction is generated, the dashboard turns into a visual summary of the ECG analysis. The status card shows whether the signal is normal or arrhythmic, while the gauge converts the risk score into a percentage in a simple visual form. A bar-style prediction breakdown shows the relative probabilities of the top classes, so the user can see how strong the model response is instead of seeing only a single label.

The ECG chart plays an important role in explainability. The plot uses the original signal values and overlays the explanation map to highlight parts of the waveform that influenced the prediction more strongly. In the implementation, points that cross a relevance threshold are shown differently, which makes it easier to compare the full waveform with suspected abnormal segments and understand where the model focused its attention.

The result interpretation is intentionally cautious. The application shows warnings when confidence is low, signal quality is poor, or uncertainty is high. In a medical decision-support setting, it is better to communicate uncertainty clearly than to present a result that looks stronger than it really is.

Testing was done at several levels. First, the preprocessing functions were checked to make sure that resampling, filtering, peak detection, and segmentation all worked correctly on common signal shapes. Second, the API was tested through upload and prediction requests. Third, the frontend was exercised by loading demo data, uploading CSV files, and generating a report.

6. Conclusion and Future Scope

The future scope of the project is strong because the codebase already includes the main building blocks needed for expansion. More datasets can be added to the unified loader, a larger training run can be done with the full data, and more advanced ensemble strategies can be used if enough computation is available. The frontend can also be extended to support historical patient records, trend analysis, and batch review of multiple signals.

In a more advanced version, the system could be connected to wearable ECG hardware or IoT devices for live monitoring. That would turn the project into a real-time screening platform instead of an offline demo. The explanation map could also be improved into a stronger visualization of attention or segment importance. Another useful extension would be a multi-user login system with role-based access for clinicians, technicians, and administrators.

In conclusion, the ECG Arrhythmia Detection System shows how a CNN-based deep learning model can be combined with preprocessing, calibration, explainability, and a web dashboard to create a complete application. The project is practical, presentable, and clearly tied to the source code. It fits well as a university report because it brings together software engineering and machine learning in one coherent workflow.

References

[1] FastAPI Documentation and API Reference:

https://fastapi.tiangolo.com/ , Accessed on Jan 28, 2026

Comprehensive guide to building APIs with FastAPI, including routing, validation, and async support

[2] PyTorch Documentation:

https://pytorch.org/docs/stable/index.html , Accessed on Feb 14, 2026

Detailed documentation for building and training neural networks using PyTorch

[3] SciPy Signal Processing Documentation:

https://docs.scipy.org/doc/scipy/reference/signal.html , Accessed on Mar 3, 2026

Resources for signal processing, filtering, and analysis using SciPy

[4] ReportLab User Guide:

https://www.reportlab.com/docs/reportlab-userguide.pdf , Accessed on Apr 10, 2026

Guide for creating PDFs programmatically using the ReportLab library

[5] React Documentation and Recharts Library:

https://react.dev/
https://recharts.org/en-US/ , Accessed on Feb 25, 2026

Official React documentation and Recharts library for building data visualizations in React applications

[6] ECG Arrhythmia Detection Repository (README & Source Code):

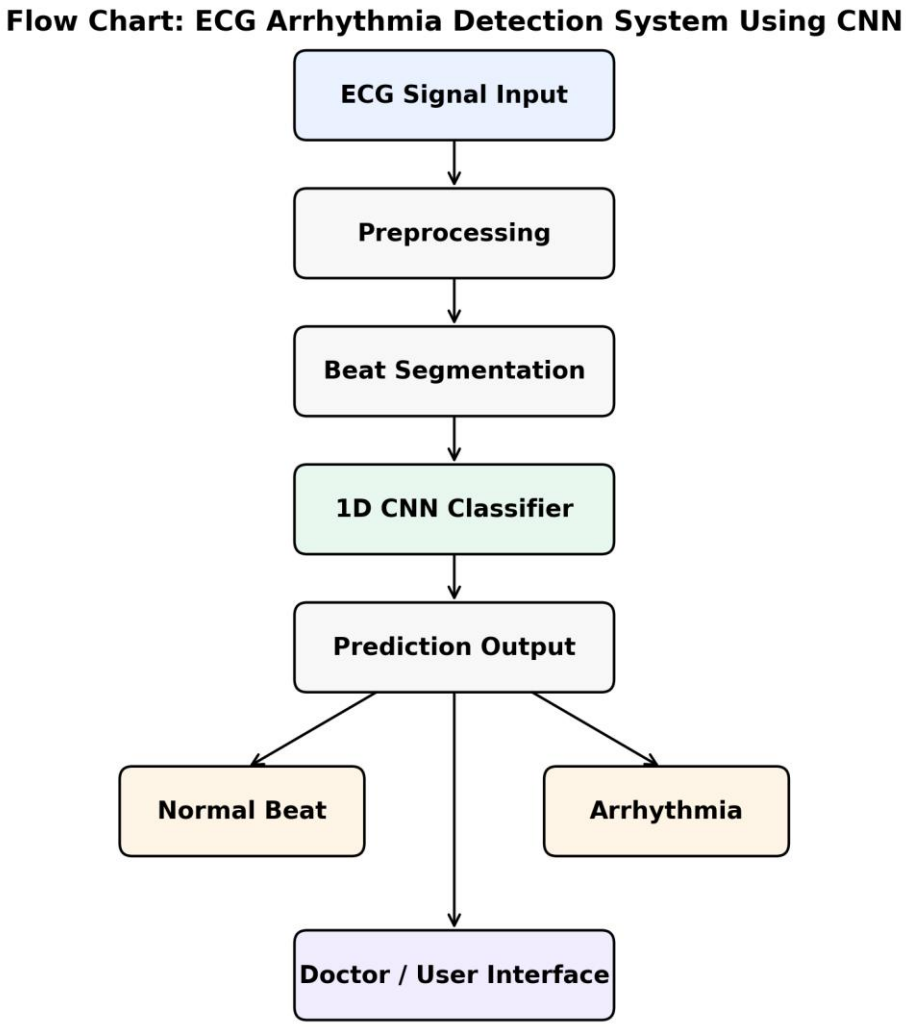
https://github.com/palaksingla2004/Arrhythmia-Detection , Accessed on Apr 18, 2026

Project repository containing implementation details, README, and source code for ECG arrhythmia detection

Appendices

Appendix A. Flowchart

The flowchart below gives a quick view of the whole system for viva or internal assessment. It shows how ECG data moves through the pipeline: the user provides the signal, the backend processes it, the model generates the output, and the frontend displays the result.



Appendix Figure: Workflow of the ECG arrhythmia detection system

Appendix B. Module-wise Summary

Module	Role in the project
Preprocessing	Cleans, normalizes, resamples, and segments ECG signals
Inference	Produces arrhythmia, risk, confidence, and explanation outputs
Frontend	Collects input and displays visual prediction results
Reporting	Creates a downloadable PDF summary
Training	Supports model training, evaluation, and artifact creation

Appendix C. Short Implementation Notes

The system expects the signal to contain enough points for the preprocessing logic to work properly. In the frontend, the user is asked to provide at least 100 values before prediction is enabled. This helps avoid empty requests and improves the stability of the demo.

The explanatory output is not meant to replace clinical reasoning. Instead, it helps the reader see which parts of the waveform were considered more important by the model. That makes the project easier to explain in a viva because the output is visible, structured, and connected to the architecture used in the code.

The project folder structure is also useful for documentation. The backend package separates data, models, training, utilities, and API routes. The frontend package separates the dashboard, chart, gauge, breakdown, and explanation components. This organization makes the system easier to describe clearly in report form.